sas innovate

Simplifying SAS® Viya®: Explaining the Compute and CAS Servers, Caslibs, and In-Memory Data

Carleigh Jo Crabtree
Technical Training Consultant

#SASinnovate

Contents

Abstract **2**

Exploring SAS Studio **3**

Demo 1: Running SAS Code on the Compute Server **4**

Demo 2: Using SAS Libraries on the Compute Server **5**

Demo 3: Creating a CAS Session and Caslib **6**

Demo 4: Exploring Caslib Attributes **8**

Demo 5: Loading Files into Memory **9**

Demo 6: Running a DATA Step in CAS **13**

- Load File into Memory 13
- Helpful Options 13
- SUM Statement Considerations 15
- Grouping Data on the CAS Server 16

HOW Slides **19**

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

Abstract

SAS® Viya® is a modern, cloud-enabled analytics platform built for high-speed, scalable processing- but what does that mean in practice for SAS® programmers? This demonstration demystifies SAS® Viya® by explaining its core architecture, introducing caslibs, and showing how they compare to traditional SAS® libraries. You'll also see how to load data into memory and explore key considerations when working with the CAS server. Whether you're new to SAS® Viya® or transitioning from SAS® 9, this session provides a practical foundation for working effectively in SAS® Viya® .

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

Exploring SAS Studio

1. Launch Google Chrome from the desktop and select **SAS Studio** from the Bookmarks bar.
2. Enter **student** in the **User ID** field and **Metadata0** in the **Password** field. Click **Sign in**. Select **Yes** when prompted to opt in to all assumable groups.
3. Click the **applications menu** icon in the upper left corner. The applications menu shows all the available applications. Depending on your environment, these applications might differ. Select **Develop Code and Flows** to open SAS Studio. We use SAS Studio to write and submit code in this workshop.
4. The Start Page tab is open by default, and it provides convenient links to start writing a program or use other point-and-click tools, including importing and querying data. Click **Program in SAS** to open a new tab with the Program Editor. Each item that you open in SAS Studio has its own tab.
5. The navigation pane is on the left side of SAS Studio. Each section in the navigation pane provides access to different resources in your SAS environment.
    - **Open Items** enables you to quickly view and access files that are currently open in your SAS Studio session. You can see that the list of open files matches the tabs.
    - **SAS Server** provides access to files and folders in your SAS environment. Keep in mind that the files and folders that you see here are not on your local machine. They're on the server where SAS is running. If you have the available permissions, you can use the **Upload files** button to load local files to the SAS server. The SAS Server also enables you to perform other tasks, such as background submit and schedule programs and basic file operations, such as copy, paste, and delete.
    - **Steps** enables you to build flows in SAS Studio. In a flow, you can build a sequence of steps, including programs, queries, and data transformations, as well as your own custom steps.
    - **Tasks** enables you to access tasks in SAS Studio. Tasks are based on SAS procedures and generate SAS code and formatted results for you.
    - **Snippets** enables you to access snippets. Snippets are lines of commonly used code or text that you can save and reuse. SAS Studio provides several predefined snippets, and you can create your own.
    - **Library Connections** provides access to your SAS libraries. You can open SAS tables and add them to your programs. You can even expand a table and view its columns.
    - **Git Repositories** enables you to access the basic Git features from within SAS Studio for maintaining and viewing a history of program edits.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

Demo 1: Running SAS Code on the Compute Server

6. In SAS Server, expand **Files** > **Home** > **workshop** > **SSV**. Double-click **Demo_01.sas**.
7. Demo 1 includes code that was originally written for SAS®9. When executed in SAS Viya, the program is processed on the Compute Server. Examine the program and notice that it includes traditional code to create a new table, and generate summary reports.

```
/* Data is loaded into and out of memory 3 times */


DATA work.cars_SUV;
     SET sashelp.cars;
     WHERE Type= "SUV";
RUN;


PROC MEANS data= work.cars_SUV;
     VAR MPG_Highway;
RUN;


PROC FREQ data= work.cars_SUV;
     TABLES Make;
RUN;
```

8. Highlight the code in Demo 1 and click **Run**.
9. This program executes on the Compute Server. It processes the data just like the SAS®9 Workspace Server and returns results for the MEANS and FREQ procedures, and an output table for the DATA step.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

Demo 2: Using SAS Libraries on the Compute Server

10. Open **Demo_02.sas** from SAS Server. Expand **Files** > **Home** > **workshop** > **SSV**.
11. Demo 2 includes a LIBNAME statement to create the **sasLib** library to connect data to the Compute server.

```
/* Library is visible in Library Connections */


LIBNAME sasLib base "/home/student/Courses/PGVY/data";


PROC FREQ data=sasLib.orders;
     TABLES Country /nocum nopercent;
RUN;
```

12. Highlight the code in Demo 2 and click **Run**.
13. This program executes on the Compute server and the **sasLib** library is visible in Library Connections as expected.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

Demo 3: Creating a CAS Session and Caslib

14. Open **Demo_03.sas** from SAS Server. Expand **Files** > **Home** > **workshop** > **SSV**.
15. To work with the CAS server, you first must create a CAS session. Highlight and run the CAS statement.

```
CAS CJsSession;
```

16. Next, data source files must be added to a caslib. Highlight and run the CASLIB statement to create the **dataCaslib** caslib.

```
CASLIB dataCaslib PATH="/home/student/Courses/PGVY/data";
```

17. Navigate to Library Connections. Notice **dataCaslib** is not visible. The caslib is available, however, it must be mapped to a libref for it to be visible in Library Connections and used in programs with the *libref.tableName* naming conventions.
18. Reset the SAS session by clicking **Options** > **Reset SAS session**.
19. To map a libref to a caslib while creating it, add the LIBREF= option on the CASLIB statement. Highlight and run the CAS and CASLIB statements.

```
CAS CJsSession;


/* Map a libref to caslib with LIBREF= */

/* The caslib is now visible in Library Connections */


CASLIB viyacas PATH="/home/student/Courses/PGVY/data"
LIBREF=viyacas;
```

20. Navigate to Library Connections. Notice **viyacas** is now listed as a library.
21. You may have predefined caslibs in your sessions that do not have a libref mapped. Use the LIBNAME statement with the CAS engine to map librefs to existing caslibs. The CASLIB statement can be used to list all available caslibs.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

22. Highlight and run the CASLIB and LIBNAME statement to assign a libref to a predefined caslib.

```
/* What if you have a caslib but it's not mapped to a libref? */
/* View all predefined caslibs */

CASLIB _all_ list;

/* Use libname statement to map libref to predefined caslib */

LIBNAME mpd CAS CASLIB=ModelPerformanceData;
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

Demo 4: Exploring Caslib Attributes

23. Open **Demo_04.sas** from SAS Server. Expand **Files** > **Home** > **workshop** > **SSV**.
24. Reset the SAS session by clicking **Options** > **Reset SAS session**.
25. Highlight and run the CAS and CASLIB statements.

```
CAS CJsSession;


/* View attributes of CASUSER caslib */


CASLIB casuser list;
```

26. In the Log, notice the **casuser** caslib is a global scope caslib (Local=No), it is the active caslib (Active=Yes), and it is only available to you (Personal=Yes).
27. Highlight and run the remaining CASLIB statements to create a new global caslib **viyacas**.

```
/* Creating a new caslib changes the active caslib */


CASLIB viyacas PATH="/home/student/Courses/PGVY/data" LIBREF=viyacas
global;


CASLIB viyaCas list;


CASLIB casuser list;
```

28. In the Log, notice **viyaCas** is now the active caslib. Creating a new caslib changes the active caslib.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

Demo 5: Loading Files into Memory

29. Open **Demo_05.sas** from SAS Server. Expand **Files** > **Home** > **workshop** > **SSV**.
30. Once data source files are connected to a caslib, they must be loaded into memory to be used in CAS. To view data source files or in memory tables, use the CASUTIL procedure with the LIST statement.
31. Highlight and run the first two CASUTIL procedures to view the files and tables currently available in **viyaCas**.

```
/* View data source files in viyaCas */


PROC CASUTIL INCASLIB="viyaCas";
     LIST files;
QUIT;


/* View in-memory tables in viyaCas */


PROC CASUTIL INCASLIB="viyaCas";
     LIST tables;
QUIT;
```

32. Notice there are no in-memory tables currently available in **viyaCas**. This is expected, as we have not loaded any data source files into memory.
33. When a data source file is already in a caslib, use PROC CASUITL to load the file into memory. This is known as a “server-side” load. In-memory tables can be session scope (only available in the current application), or global scope (available in other applications like SAS Visual Analytics). The following PROC CASUTIL steps load orders.csv into memory. In the second PROC CASUTIL, the PROMOTE option is added to promote the table to global scope.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

34. Highlight and run the PROC CASUTIL steps.

```
/* Load orders.csv to a session scope in-memory table */


PROC CASUTIL;
   LOAD CASDATA="orders.csv"
   INCASLIB="viyacas"
   CASOUT="ordersIM"
   OUTCASLIB="viyaCas";
QUIT;


/* Load orders.csv to a global scope in-memory table by adding
PROMOTE */
/* Table is visible in VA */


PROC CASUTIL;
   LOAD CASDATA="orders.csv"
   INCASLIB="viyacas"
   CASOUT="ordersIM_global"
   OUTCASLIB="viyacas"
   PROMOTE;
QUIT;


PROC CASUTIL INCASLIB="viyaCas";
     LIST tables;
QUIT;
```

35. Navigate to SAS Visual Analytics by clicking Applications menu > Explore and Visualize. Click New report > Add data. Notice **ordersIM_global** is available within the application. **OrdersIM** is not available because it is session scope.

36. When a data source file is not already in a caslib, but you want to use it in CAS, the file can be loaded into memory in a caslib in a number of ways. This is known as a “client-side” load.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

37. When loading a client-side SAS dataset, use the DATA= option on the LOAD statement. The following PROC CASUTIL loads SASHELP.CARS into the **viyaCas** caslib as the global scope in-memory table **cars_cas**. Highlight and run the steps.

```
/* Load SAS data set using PROC CASUTIL */


PROC CASUTIL;
   LOAD DATA=sashelp.cars
   CASOUT="cars_CAS"
   OUTCASLIB="viyacas"
   PROMOTE;
QUIT;


PROC CASUTIL INCASLIB="viyacas";
     LIST tables;
QUIT;
```

38. When loading a client-side CSV file, use the FILE= option on the LOAD statement. The following PROC CASUTIL loads airport_traffic_2020.csv into the **viyaCas** caslib as the global scope in-memory table **EU_Air_2020**. Highlight and run the steps.

```
/* Load CSV file using PROC CASUTIL */


PROC CASUTIL;
   LOAD FILE="/home/student/workshop/SSV/airport_traffic_2020.csv"
   CASOUT="EU_Air_2020"
   OUTCASLIB="viyacas"
   PROMOTE;
QUIT;


PROC CASUTIL INCASLIB="viyacas";
     LIST tables;
QUIT;
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

39. A SAS dataset can also be loaded into memory in a DATA step. The libref of the output dataset must specify a caslib. Add the PROMOTE=YES data set option to promote the dataset to global scope. The DATA step will run on the Compute server, then load the file into memory. The following DATA step reads and filters SASHELP.CARS, and creates the global scope in-memory table **VIYACAS.ToyotaCars_CAS**. Highlight and run the steps.

```
/* Load and promote SAS data set using DATA step */


DATA viyacas.ToyotaCars_CAS (PROMOTE=YES);
   SET sashelp.cars;
   WHERE Make="Toyota";
RUN;


PROC CASUTIL INCASLIB="viyacas";
     LIST tables;
QUIT;
```

40. A CSV file can also be loaded into memory using PROC IMPORT. The libref of the output dataset on the OUT= option must specify a caslib. The following PROC IMPORT reads the airport_traffic_2022.csv file and creates the in-memory table VIYACAS.EU_AIR_2022. Highlight and run the steps.

```
/* Load CSV using PROC IMPORT */


PROC IMPORT
DATAFILE="/home/student/workshop/SSV/airport_traffic_2022.csv"
            DBMS=csv
            OUT=viyacas.EU_Air_2022
            REPLACE;
RUN;


PROC CASUTIL INCASLIB="viyacas";
     LIST tables;
QUIT;
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

Demo 6: Running a DATA Step in CAS

41. Open **Demo_06.sas** from SAS Server. Expand **Files** > **Home** > **workshop** > **SSV**.
42. When working with data on the CAS server, there are some helpful options and modifications needed when working with the DATA step.

Load File into Memory

43. Highlight and run the CASLIB, CASUTIL procedures to create a new global caslib **ssvCas** and load the chocolate_enterprise_reporting.sas7bdat file into memory as **SSVCAS.Choc_Enterprise**.

```
CASLIB ssvCas PATH="/home/student/workshop/SSV" LIBREF=ssvCas
global;


proc casutil;
    list files;
quit;


PROC CASUTIL;
    /*data source file name*/
    LOAD CASDATA="chocolate_enterprise_reporting.sas7bdat"
    /*caslib the data source file is currently in*/
    INCASLIB="ssvcas"
    /*name of the new in-memory table*/
    CASOUT="Choc_Enterprise"
    /*caslib to put the in-memory table in*/
    /*can be the same or different from the INCASLIB*/
    OUTCASLIB="ssvcas"
    promote;
QUIT;
```

Helpful Options

44. Use the SESSREF option to specify to only run a DATA step in CAS. If it is not fully CAS-enabled, the step will stop and the Log will produce an error.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

45. Highlight and run the first DATA step. Notice the libref of the output dataset is WORK, which is not a caslib. The DATA step is not fully CAS-enabled and does not execute. An error is produced in the Log.

```
/* Output dataset is not in a caslib */

/* Sessref option prevents the step from running on Compute */

data work.EU_Orders /sessref= CJsSession;

     set ssvcas.choc_enterprise;

     where continent="Europe";

run;
```

46. Highlight and run the second DATA step. Notice the libref of the output dataset is the caslib SSVCAS. The DATA step is fully CAS-enabled and executes on the CAS server.

```
/* Output dataset is now a caslib so the step can run in CAS */

data ssvcas.EU_Orders /sessref=CJsSession;

     set ssvcas.choc_enterprise;

     where continent="Europe";

run;
```

47. The MSGLEVEL option specifies the level of detail in messages that are written in the Log. Specify i for “in-depth” messages, or n for “normal” messages. Highlight and run the OPTIONS and DATA step.

```
options msglevel=i;


data ssvcas.EU_Orders /sessref=CJsSession;

     set ssvcas.choc_enterprise;

     where continent="Europe";

run;
```

48. Use PUT _THREADID_ = _N_ to visualize how data is distributed among threads. Highlight and run the DATA step. In the Log, notice how the data was distributed.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

```
data ssvcas.EU_Orders;
     set ssvcas.choc_enterprise end=eof;
     where continent="Europe";
     if eof then put _threadid_= _N_=;
run;
```

SUM Statement Considerations

49. Because the CAS server is multithreaded, the results returned from the SUM statement may not be as expected. For example, in the following DATA step, the data is distributed to multiple threads. The number of threads determines how many rows of data are returned, rather than one row as expected. Highlight and run the step.

```
data ssvcas.EU_Orders;
     set ssvcas.choc_enterprise end=eof;
     where continent="Europe";
     if country_nm="France" then FranceOrders+1;
     if eof=1 then output;
     keep FranceOrders;
     if eof then put _threadid_= _N_=;
run;
```

50. To fix this there are two solutions using SINGLE=YES on the DATA statement. Solution 1 is to run a second, single threaded DATA step using the previously summarized dataset. This is the most efficient option if you are working with large data. Highlight and run the DATA step.

```
data ssvcas.EU_Orders_Sum / single=yes;
     set ssvcas.eu_orders end=eof;
     TotalFranceOrders+FranceOrders;
     keep TotalFranceOrders;
     if eof=1 then output;
     if eof then put _threadid_= _N_=;
run;
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

51. Solution 2 is to run one single threaded DATA step in CAS. Depending on the size of your data, this may effect efficiency. Highlight and run the DATA step.

```
data ssvcas.EU_Orders_Single / single=yes;
     set ssvcas.choc_enterprise end=eof;
     where continent="Europe";
     if country_nm="France" then FranceOrders+1;
     if eof=1 then output;
     keep FranceOrders;
     if eof then put _threadid_= _N_=;
run;
```

52. Remember, running the DATA step on the Compute server is also an option.

Grouping Data on the CAS Server

53. When grouping data on the CAS server, there is no need to presort data, and each group remains together on a thread. The first BY variable determines how the data is distributed to the threads.
54. Highlight and run the PROC SQL step. Notice there are four distinct values in the **Country_NM** variable.

```
proc sql;
select distinct country_nm
     from ssvcas.choc_enterprise;
quit;
```

55. The following DATA step counts the total number of orders placed in each country. Notice **Country_NM** is used to group data on the BY statement. Highlight and run the step. Each country's data is distributed to a thread, only four threads are used.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

```
data ssvcas.choc_country;
     set ssvcas.choc_enterprise end=eof;
     by country_nm;
     if first.country_nm then CountryTotal=0;
     CountryTotal+1;
     if last.country_nm then output;
     keep country_nm CountryTotal;
     if eof then put _threadid_= _N_=;
run;
```

56. When there are multiple BY variables, row order within a group can vary. Highlight and run the following DATA step twice. Make note of the **customer_name** in the first row each time. Notice it changes.

```
data ssvcas.choc_city;
     set ssvcas.choc_enterprise end=eof;
     by country_nm city_nm;
     keep country_nm city_nm transaction_id customer_name;
     if eof then put _threadid_= _N_=;
run;
```

57. To guarantee row order within a group, use the ADDROWID=YES data set option when loading a file into memory. This adds a variable **_ROWID_** to the data that numbers each row. Highlight and run the LIBNAME and DATA step to create a table with the **_ROWID_** variable.

```
libname choc "/home/student/workshop/SSV";


data ssvcas.choc_rowid (addRowId=yes);
     set choc.chocolate_enterprise_reporting;
run;
```

58. In the DATA step, notice the _ROWID_ variable is added as the last variable on the BY statement. Highlight and run the DATA step twice and notice row order within the group is now consistent.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

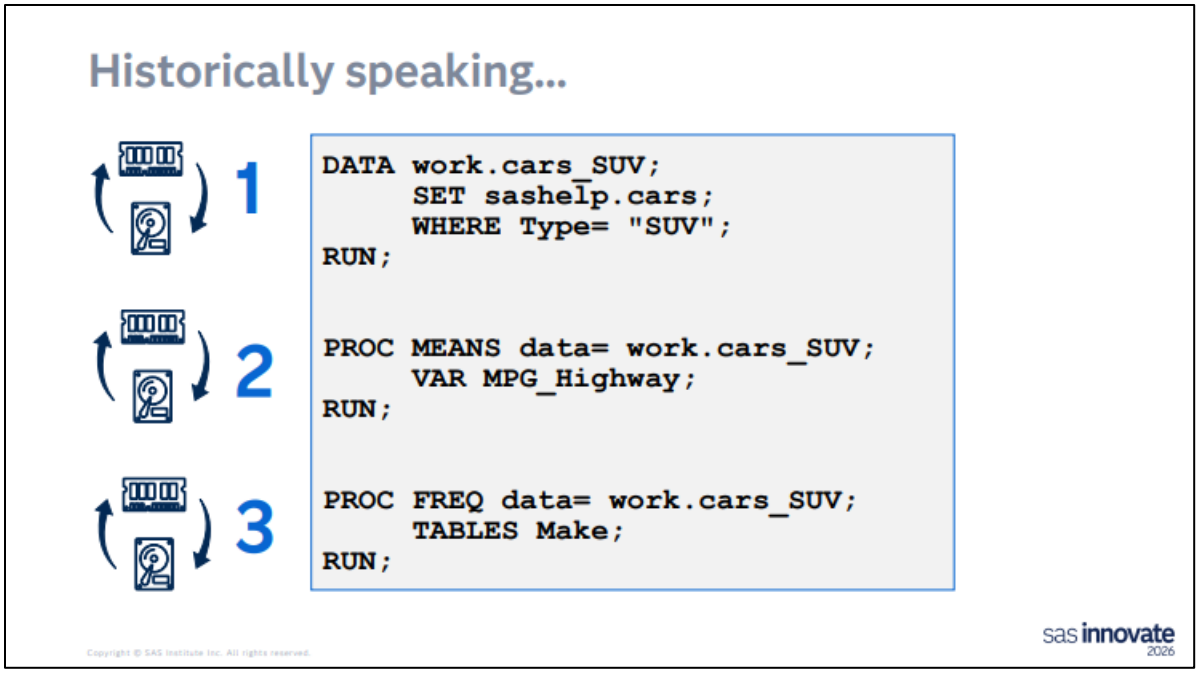
```
data ssvcas.withrowid;
     set ssvcas.choc_rowid end=eof;
     by country_nm city_nm _rowid_;
     keep country_nm city_nm transaction_id customer_name _rowid_;
     if eof then put _threadid_= _N_=;
run;
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

HOW Slides

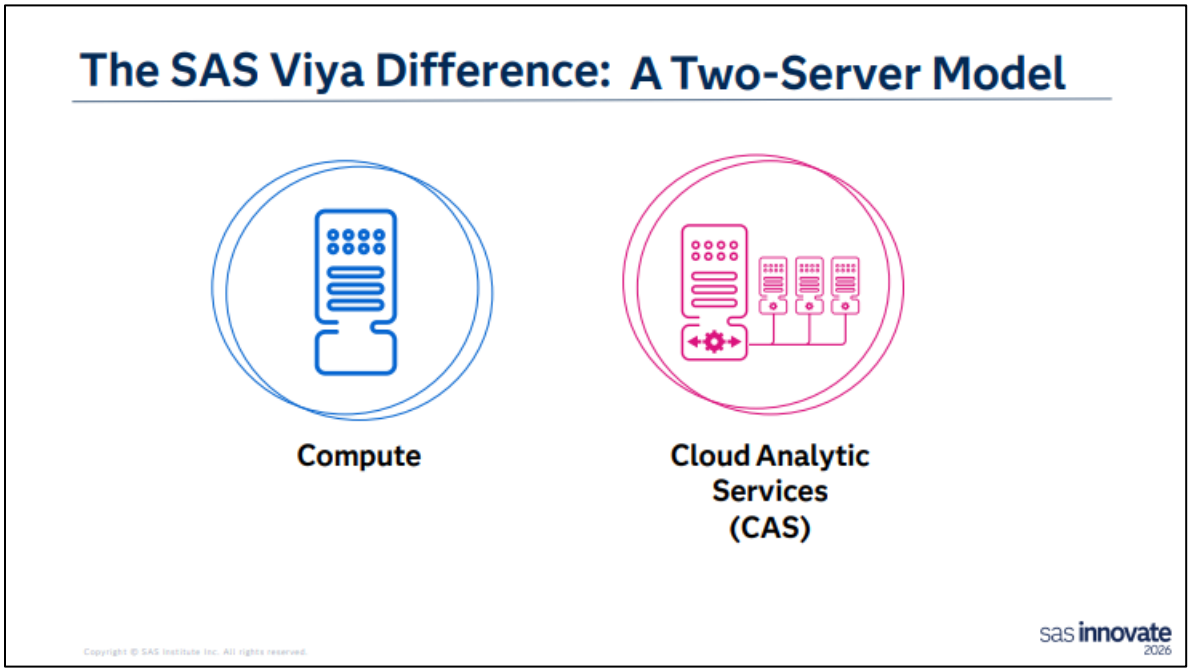
SAS Viya is a modern, cloud-enabled analytics platform built for high-speed processing and scalability. Throughout this presentation, we'll cover the two main servers, how caslibs work, and how to process in-memory data. Let’s get started!

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.



If this program was run in SAS 9, the data would be loaded into and out of memory for each step which leads to increased input/output (I/O).

This is manageable for small programs but becomes very time-consuming when working with large datasets or complex processes.

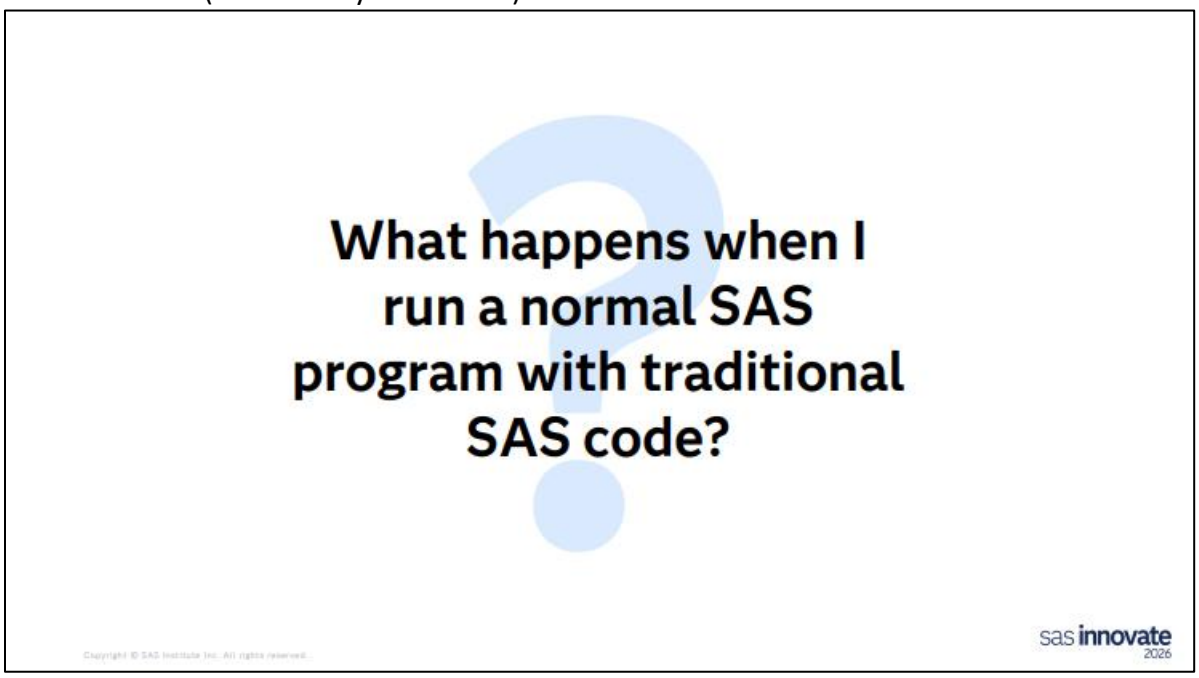


In SAS Viya, there are two primary servers, providing options to programmers depending on the processing demands of their programs.

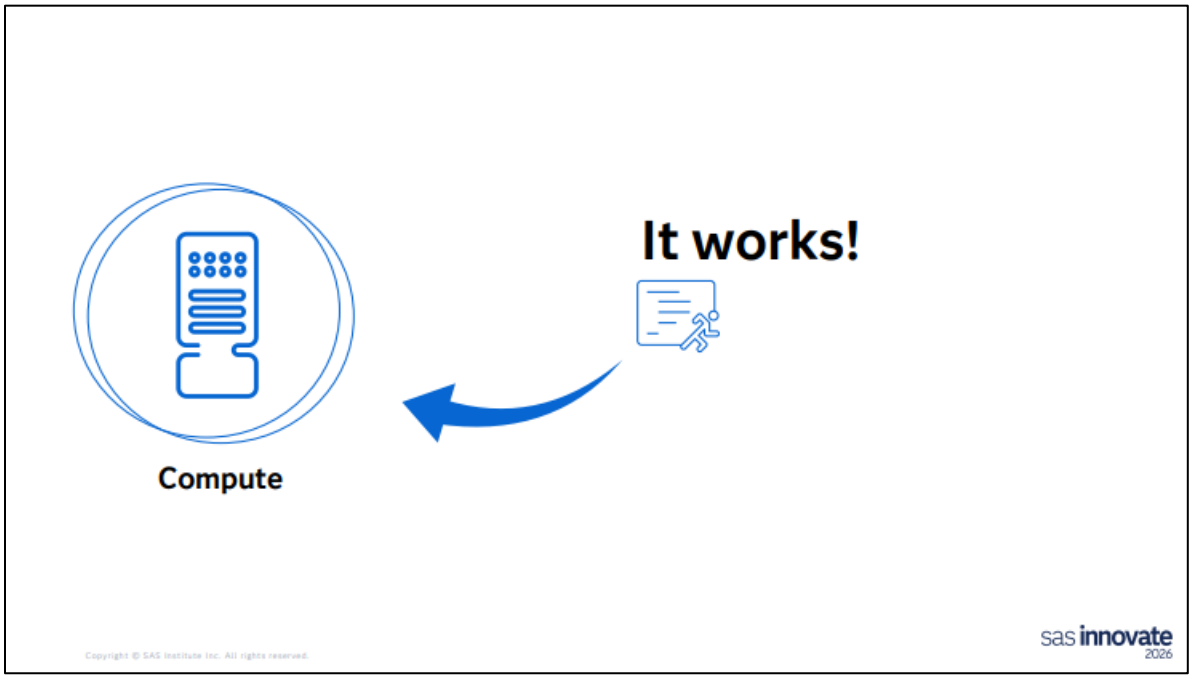
The two servers are:

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

- SAS Compute Server
- CAS (Cloud Analytic Services) Server



What happens when I run a normal SAS program with traditional SAS code?

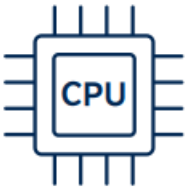


It works! The Compute Server works similarly to the traditional SAS 9 Server where data is read into and cleared from memory with each step. You can run your existing SAS programs with normal SAS code in SAS Viya and it will execute on the Compute Server.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

It's a great option for smaller datasets or if you want to transition existing code into SAS Viya with minimal modifications.

Threads Explained



Cores	Physical **processing units** that **execute** instructions
Threads	Virtual sequence of **instructions** that can run on a single core

sas innovate 2026

In SAS Viya, some processes are single threaded and others are multithreaded. When we talk about threads we are working with a CPU. CPUs have cores which are the physical processing units that execute instructions independently. Threads are the virtual sequences of instructions that can run on a single core.

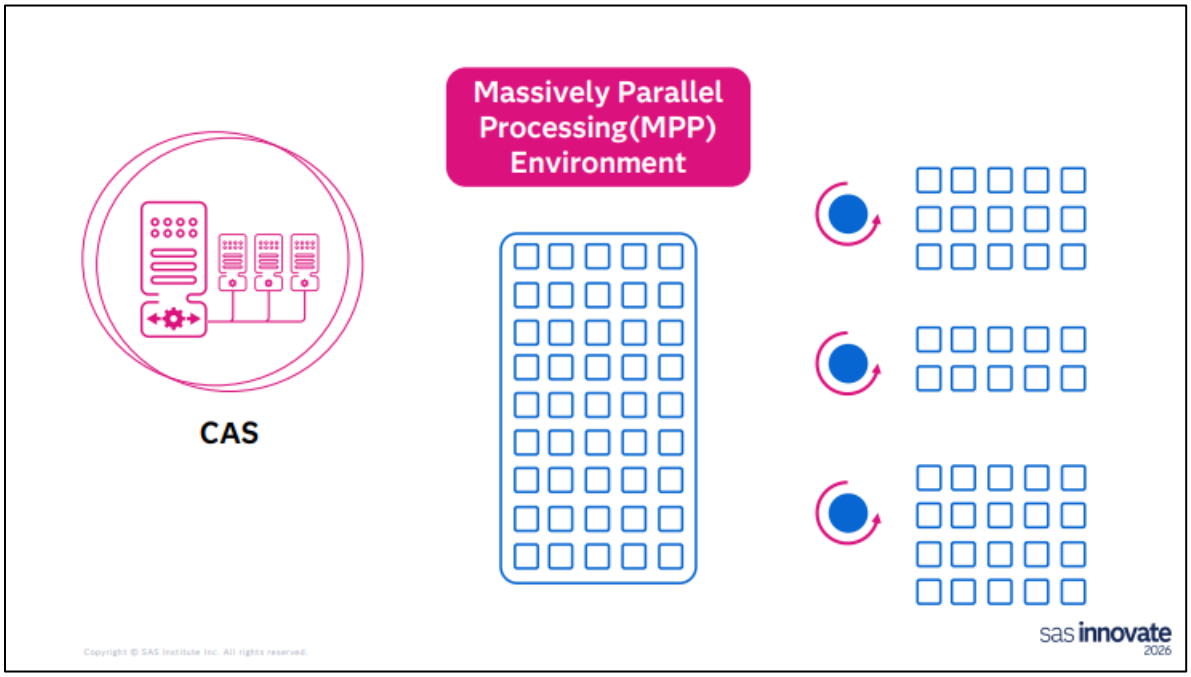
Single-Threaded Processes	Multi-Threaded Processes
execute	Allow multiple parts of a program to run concurrently
one	Different threads execute different tasks simultaneously
instruction	Allow memory and files to be shared and remains responsive even if part of it is blocked or performing a lengthy operation
at	
a	
time	

sas innovate 2026

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

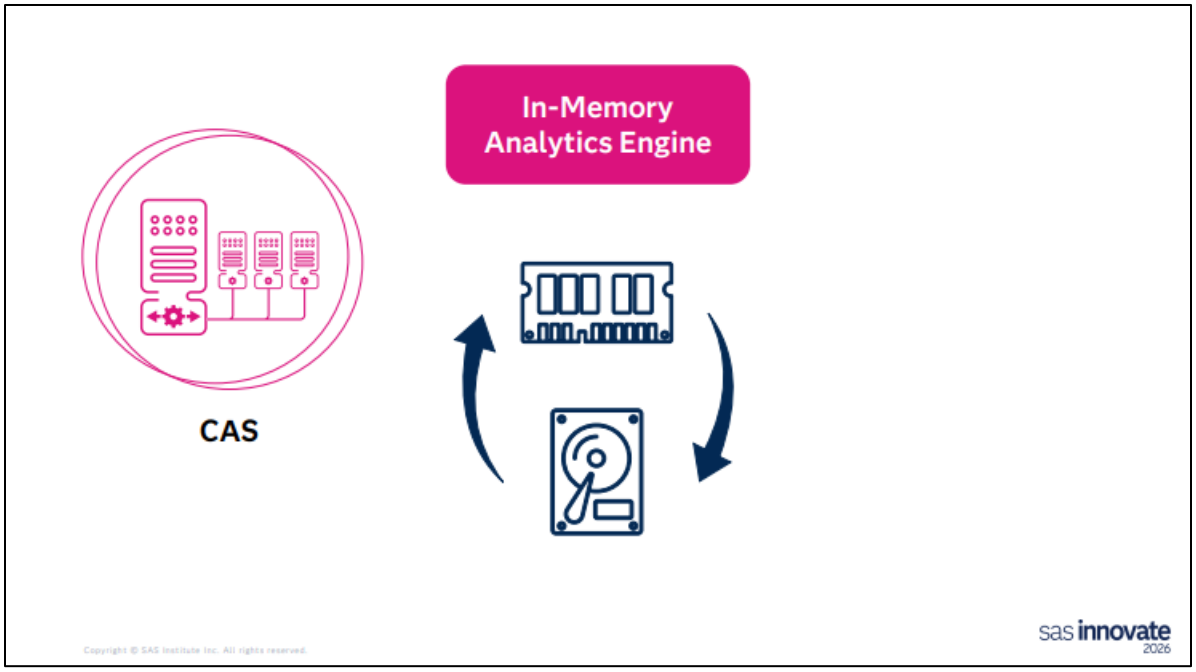
Single-threaded processes execute instructions one at a time. This can be less efficient for tasks that can be parallelized or require waiting for I/O operations.

Multi-threaded processes allow multiple parts of a program to run concurrently. Different threads can execute different tasks simultaneously, which is more efficient. Multi-threading allows memory and files to be shared, and it can remain responsive even if part of it is blocked or performing a lengthy operation.

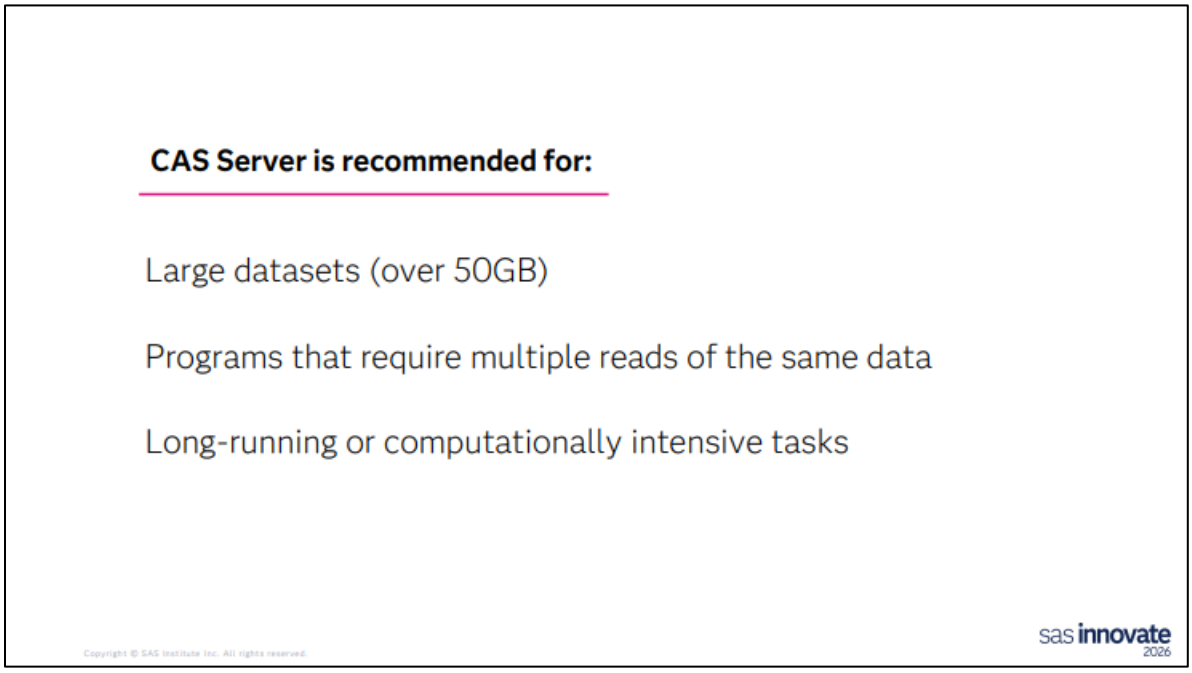


The CAS Server is a fully multi-threaded, massively parallel processing environment, meaning CAS divides large datasets into chunks and processes them across multiple nodes simultaneously. This allows programs to run faster, as tasks are being distributed.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.



Instead of loading and unloading data multiple times like in SAS 9, CAS allows you to load a dataset into memory once and keep it there until your work is finished. This greatly reduces I/O, making processing more efficient.



The CAS Server shines when you're working with:

- Large datasets (over 50GB)
- Programs that require multiple reads of the same data
- Long-running or computationally intensive tasks

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

Traditional SAS Libraries

```
LIBNAME sasLib base "path";

PROC FREQ data=sasLib.orders;
     TABLES Country /nocum nopercent;
RUN;
```

sas innovate 2026

In SAS 9, libraries are created with the LIBNAME statement and connect to data sources such as databases, cloud data, or folder paths. This is still true when working with data on the Compute server in SAS Viya.

Working with In-Memory Data

CAS Session	Create a CAS session to process in-memory data. The session encapsulates or **"keeps a record"** of our work and **connects** us to the CAS Server.

sas innovate 2026

To process data in-memory, we must first create a CAS session. The session encapsulates or "keeps a record" of our work and connects us to the CAS Server.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

Working with In-Memory Data

CASLIB	Connect to data source files and load tables into memory for processing in CAS **library : SAS 9** **caslib : CAS server in SAS Viya**

sas innovate 2026

To process data in-memory, we must first create a CAS session. The session encapsulates or “keeps a record” of our work and connects us to the CAS Server.

Create a caslib and map a libref to the caslib:

```
CASLIB caslibName PATH="pathToData" LIBREF=libName;
```

Map a libref to existing caslib:

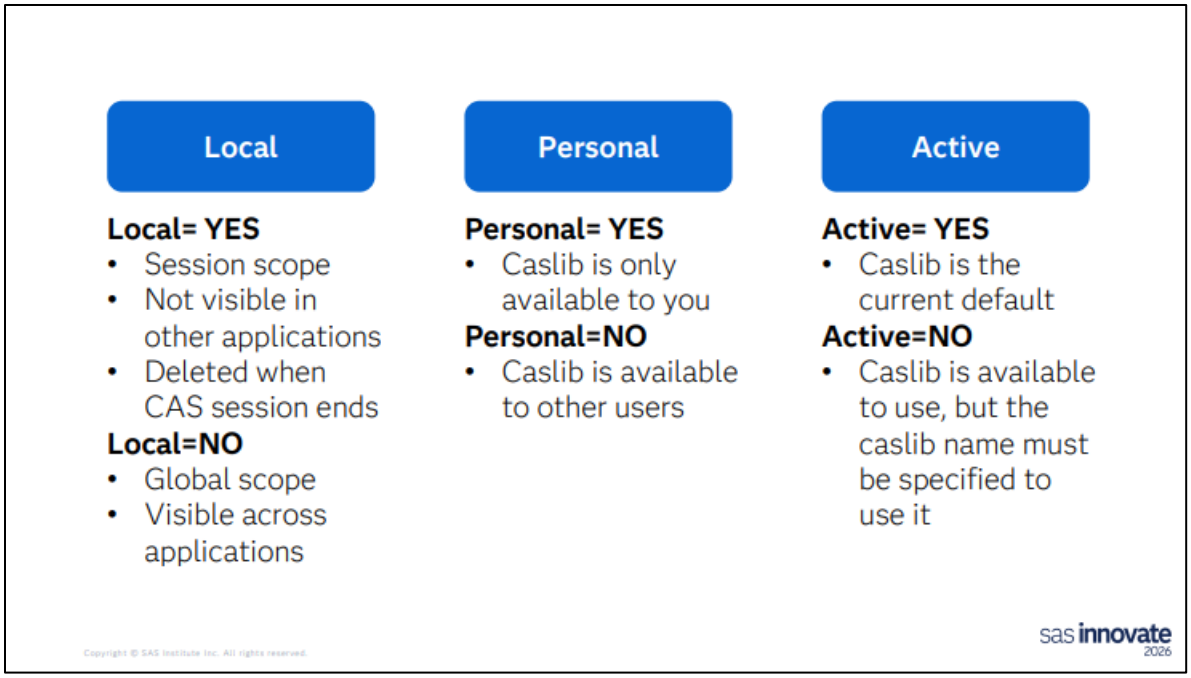
```
LIBNAME libName CAS CASLIB=caslibName;
```

sas innovate 2026

To create a caslib and map a libref to the caslib, use the caslib statement.

To map a libref to an existing caslib, use a libname statement with the CAS engine.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.



Caslibs have attributes that describe their data connection and user access. The three main attributes are Local, Personal and Active.

Local determines the scope of a caslib. If local=yes, the caslib is “session scope” meaning it is not visible in other applications like SAS Visual Analytics. When the CAS session ends, the caslib is deleted.

- SAS 9 comparison: Traditional SAS libraries are deleted at the end of a SAS session. You must run a LIBNAME statement to work with your data again.

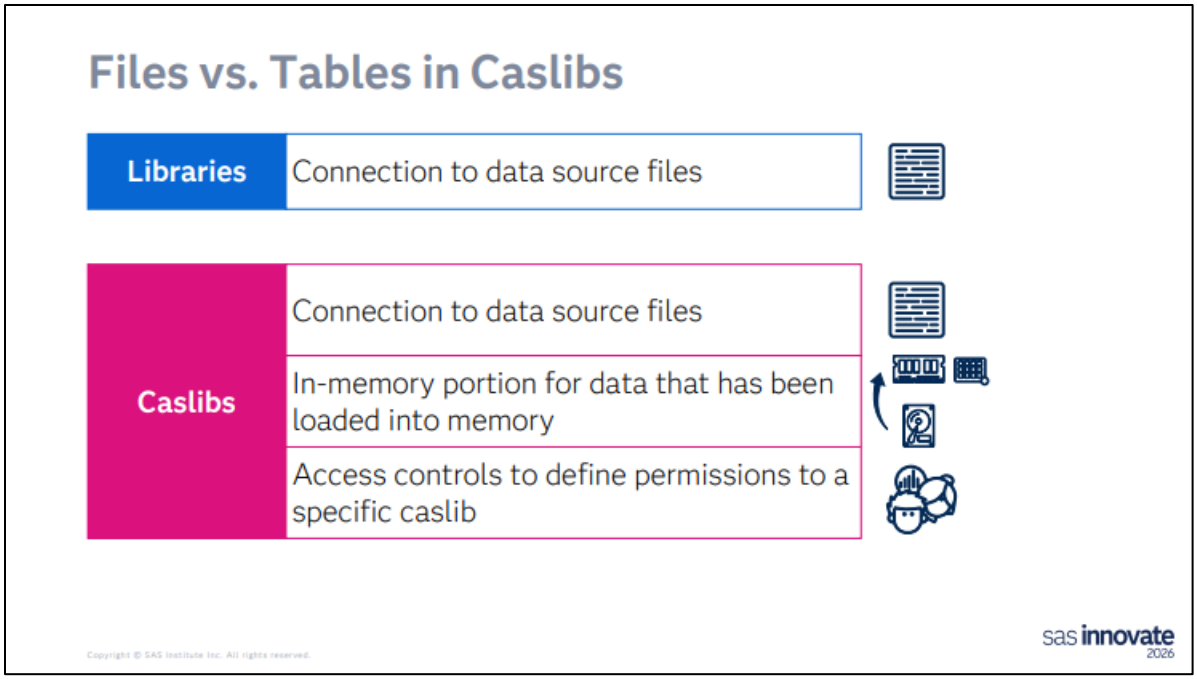
If local=no, the caslib is “global scope”. Anyone with permission can access it, and it's visible across applications. When the CAS session ends, the caslib is not deleted. Global scope is useful when sharing data across sessions, with other users and when working with large data that you do not want to load and unload from memory often.

- SAS 9 comparison: Traditional SAS has predefined libraries such as SASHELP that persist across SAS sessions.

Personal determines if the caslib is available to only you, or available to other users.

Active determines the "default" caslib. In traditional SAS, the work library is the default when a library is not specified in a program. The CAS version of the work library is the “active” caslib. The active caslib can change. It is typically casuser by default, however, if you have just created a caslib, that new caslib will become the active caslib. It is best practice to specify the caslib you are working with.

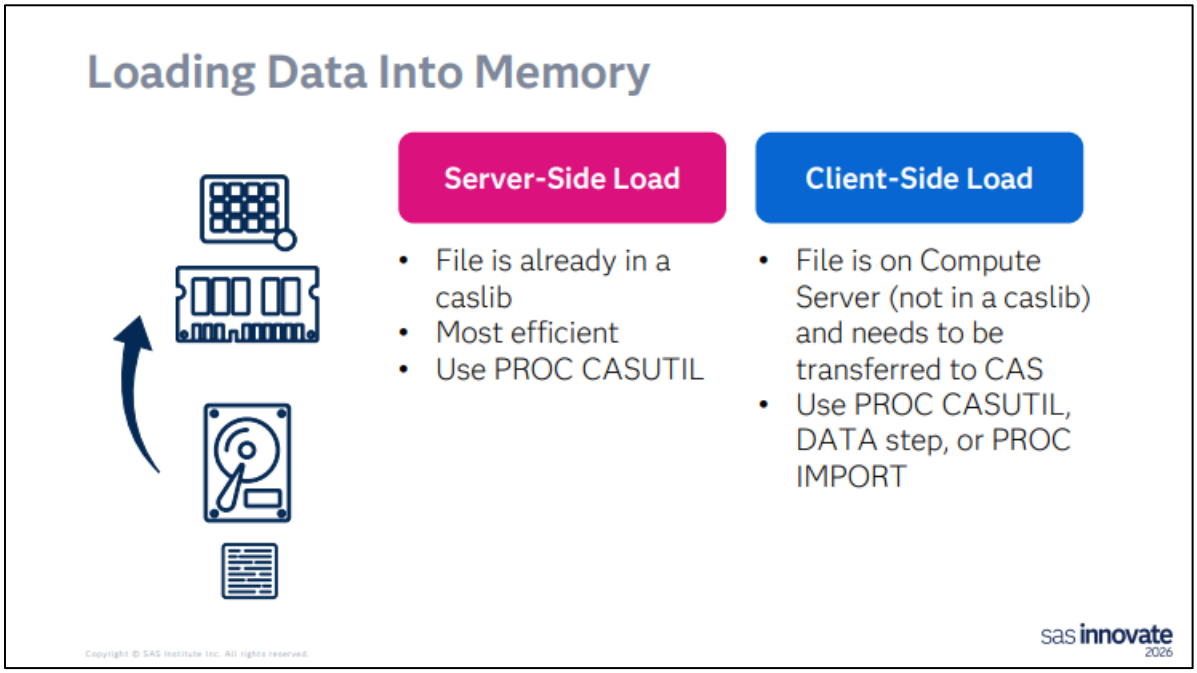
Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.



Traditional SAS libraries only connect to data source files.

Caslibs have three main components:

1. The connection to the data source files.
2. The in-memory portion for data that has been loaded into memory.
3. Access controls to define permissions to a specific caslib.



To work with data on the CAS server, you must load data source files to in-memory tables. This can be done using either a server side load or a client-side load.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

- Server-Side Load: Use when the file is already in a caslib. This is the most efficient method and is performed using PROC CASUTIL.
- Client-Side Load: Use when the file is on the Compute Server and needs to be transferred to CAS. This can be done using PROC CASUTIL, the DATA step, or PROC IMPORT.

```
PROC CASUTIL;
     LIST FILES | TABLES;
QUIT;
```

sas innovate 2026

The CASUTIL procedure is a utility procedure that can be used for a number of tasks. We will use it to view available data source files and in-memory tables in caslibs and load files into memory.

Before running a DATA step on the CAS server...

1. Start a CAS session
2. Ensure there are in-memory tables in a caslib mapped to a libref
3. Input and output tables must specify a caslib and the DATA step must be fully CAS enabled

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

Once data has been loaded into memory, it becomes available for use on the CAS server. This means in-memory data in a caslib can be accessed using the DATA step, procedures, or in other SAS Viya applications.

Before running a DATA step on the CAS server: 1. Start a CAS session. 2. Ensure there are in-memory tables in a caslib connected to a libref. After this has been done, both the input and output tables must specify a caslib and the DATA step must be fully CAS enabled. Some restrictions can cause the DATA step to run on Compute instead of CAS.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.